

SIH26140 — End-to-End Vibe Coding Build Guide

AI-Based Interactive Quantum Algorithm Learning Platform

A step-by-step process for building the full prototype using free tools and AI-assisted coding (Claude Code / Cursor).

Team stack assumed: Pure CSE/software team — Python, React, basic ML. No hardware, no domain expert.

Cost: \$0 — every tool and API used in this guide is free tier.

1. What is Vibe Coding (and when to trust it)

Vibe coding means describing what you want in plain language to an AI coding assistant (Claude Code, Cursor, etc.) and letting it write the implementation, while you review, test, and steer each step. It works best for glue code, API orchestration, UI components, and prompt engineering — tasks where the logic is well-understood but writing it by hand is slow.

Rule of thumb: vibe code the plumbing (APIs, UI, data flow). Verify by hand anything with exact correctness requirements — e.g. quantum gate math must match Qiskit's own reference outputs.

2. Tools You Need (all free)

- **Claude Code or Cursor** — the AI coding assistant that will do most of the typing
- **Groq** (console.groq.com) — free LLM API for the AI tutor, no credit card needed
- **Ollama** (ollama.com) — optional fully-offline backup LLM, zero internet dependency for demo day
- **Qiskit** — free open-source Python library for quantum circuit simulation (no real quantum hardware needed)
- **Python 3.10+, Node.js** — runtimes for backend and frontend
- **GitHub** — free repo + free Pages/Render/Railway free-tier hosting for the demo

3. Full Architecture Recap

Four components, all software, all free to run:

- **Qiskit simulator** — runs quantum circuits and algorithms on CPU
- **AI tutor engine** — classifies student answers and re-teaches adaptively (Groq API)
- **React frontend** — circuit builder + lesson UI + chat interface
- **Teacher dashboard** — shows where students struggle most

4. Phase-by-Phase Build Plan

Phase 0 — Setup (30 min)

- Create a GitHub repo for the project
- Sign up at console.groq.com, generate a free API key, save it as an environment variable
- Install Python, Node, and Qiskit locally: `pip install qiskit fastapi uvicorn openai --break-system-packages`

Phase 1 — AI Tutor Backend (Day 1)

This is the adaptive re-teaching engine — your #1 demo moment. Build this first.

- Endpoint takes a student's answer, sends it to Groq's free LLM with a grounded system prompt
- LLM classifies the answer as correct / partial / incorrect
- If not correct, LLM generates a simpler re-explanation using a different analogy
- Track per-session history in memory (upgrade to SQLite later if time allows)

Prompt to give Claude Code:

```
Build a FastAPI backend with a POST /tutor/evaluate endpoint. It takes {session_id, question, student_answer}, sends it to Groq's free API (model: llama-3.3-70b-versatile) with a system prompt that classifies the answer as correct/partial/incorrect and, if not correct, returns a simpler re-explanation using a different analogy. Track conversation history per session_id in memory. Return JSON.
```

Test immediately after this phase — ask 3 sample questions (superposition, entanglement, Grover's algorithm) and read the raw responses before moving on.

Phase 2 — Qiskit Quantum Simulator (Day 1-2)

This is the part that needs the most manual verification — do not fully vibe-code without checking outputs.

- Implement 2-3 algorithms: Deutsch-Jozsa, Grover's search (start simple)
- Build an endpoint that runs a circuit and returns the measurement results
- Cross-check every output against Qiskit's own official textbook examples

Prompt to give Claude Code:

```
Add a /simulate/grover endpoint to the FastAPI app. Use Qiskit to build a 3-qubit Grover's search circuit for a given marked state, run it on the AerSimulator, and return the measurement counts as JSON. Include a docstring explaining each step of the circuit.
```

Verify: run the same circuit directly from the official Qiskit textbook example and confirm your endpoint's output distribution matches. Do not skip this check.

Phase 3 — React Frontend (Day 2-3)

- Lesson view: question + student answer input + submit button
- Circuit visualizer: simple drag-and-drop or preset circuit diagrams calling the simulator endpoint
- Chat-style display for the AI tutor's classification and re-explanation

Prompt to give Claude Code:

Build a React component with a text area for a student's answer to a displayed question, a submit button that calls `POST /tutor/evaluate`, and a result panel showing the classification (with a color badge: green for correct, amber for partial, red for incorrect) and the re-explanation text if present.

Phase 4 — Teacher Dashboard (Day 3)

- Aggregate all sessions' history
- Show which questions/topics have the highest incorrect-answer rate
- Simple bar chart is enough — do not over-engineer this

Prompt to give Claude Code:

Add a `GET /tutor/analytics` endpoint that aggregates all sessions and returns counts of correct/partial/incorrect per question. Build a simple React dashboard page that fetches this and renders a bar chart using `recharts`.

Phase 5 — RAG Grounding (stretch goal, only if time allows)

Reduces hallucination risk by grounding the tutor's explanations in real textbook content instead of free generation.

Prompt to give Claude Code:

Add a simple RAG layer: load the Qiskit textbook's quantum algorithms chapter as text chunks, embed them with a free sentence-transformers model, and retrieve the most relevant chunk to include as context before calling the LLM in `/tutor/evaluate`.

Phase 6 — Testing Checklist

- Run at least 10 different student answers per question, including deliberately wrong ones
- Confirm re-explanations are actually different from the original lesson wording (not repeats)
- Confirm Qiskit simulator outputs match official reference examples
- Test with no internet (if using Ollama backup) to confirm offline fallback works
- Load-test with 2-3 browser tabs open simultaneously to catch session-mixing bugs

Phase 7 — Demo Day Prep

- Pre-load 2-3 guaranteed-good demo answers you know produce a clean result
- Have a backup video recording of the demo in case of live failure
- Print the differentiation talking points below — memorize the one-line pitch
- Charge laptop fully; disable auto-updates/notifications before presenting

5. Risks and Mitigations

Risk	Mitigation
LLM API cost/rate limits during demo	Groq free tier is generous; cache repeated question responses
Wrong correct/partial/incorrect classification	Add few-shot examples in the system prompt for common quantum answers
Hallucinated physics explanations	Ground with RAG (Phase 5) using real Qiskit textbook content
Qiskit circuit math errors	Manually cross-check every algorithm against official Qiskit examples
Live demo internet failure	Keep Ollama offline fallback ready and tested beforehand
Judge says 'this already exists'	Lead with the differentiation talking points (Section 6) immediately

6. Judge Q&A; Prep — Differentiation Talking Points

If a judge says: *"CUDA-Q Academic / Qiskit textbook / Microsoft Quantum Katas already do this."*

Your answer:

"Those platforms are static, English-only, and often GPU-dependent. None of them watches a student's mistake in real time and re-teaches instantly with a different analogy. None of them gives a teacher a dashboard showing exactly where a class is struggling. We built the missing personalization and classroom-deployment layer on top of the same open quantum-computing foundations they use."

Keep this list of concrete gaps ready to name specifically if pressed further:

- Real-time adaptive re-teaching (not in existing static platforms)
- Teacher-facing mistake-pattern analytics
- Runs on CPU only — no GPU infrastructure required, accessible to any college
- Built for India's National Quantum Mission skilling goals

7. Final Pre-Submission Checklist

- ☐ AI tutor endpoint working and tested with 10+ sample answers
- ☐ Qiskit simulator outputs verified against official reference examples
- ☐ Frontend connects to both endpoints without errors
- ☐ Teacher dashboard shows at least one meaningful chart
- ☐ One-line pitch memorized by every team member
- ☐ Differentiation talking points printed/memorized
- ☐ Demo tested end-to-end at least 3 times before presenting
- ☐ Backup demo video recorded
- ☐ Offline fallback (Ollama) tested if relying on venue WiFi

This guide is a build plan, not a substitute for understanding your own code. Be ready to explain any part of it in your own words — judges will ask.